

הערכה עצמית – PDF Attendance Report Generator

בדקי את עצמך אחרי שסיימת לכתוב את הפרויקט | סה"כ 110 נקודות

שם מלא:

אילה הרדי

קישור לפרויקט ב-GitHub:

<https://github.com/AyalaOnGit/Attendance-Report-Gen/tree/upgrade>

22 מתוך 22 שאלות נענו

20 נקודות

1 מבנה הפרויקט וחלוקה לשכבות

1. איך חילקת את הקוד שלך לקבצים? בחרי את התיאור שהכי מתאים לפרויקט שלך

☐ א הכל בקובץ אחד – main.py עושה הכל

☐ ב 2-3 קבצים בלי חלוקה ברורה של אחריות

☒ ג 4 אחריות נפרדות: מחלקות נתונים / קריאת PDF / לוגיקת שינוי / כתיבת פלט

☐ ד יש לי קבצים נפרדים אבל הם מייבאים אחד את השני בצורה מעגלית

2. מי מייבא את מי בפרויקט?

א ☐ כל קובץ מייבא מכל קובץ – אין חוקיות

ב ☒ תזרים חד- `app.py` `domain` `← parsing` `← transformation` `domain` כיווני:

ג ☐ ה-`parser` מייבא מה-`renderer` כדי לבדוק פורמט

ד ☐ יש לי רק קובץ אחד אז אין `imports` בין קבצים

3. איפה מוגדרים המספרים של חוקי הטרנספורמציה

(למשל: שעות עבודה מותרות, כמה דקות jitter, סף שעות נוספות)?

א ☒ במקום אחד – קובץ `config` או `dataclass` קפוא של חוקים

ב ☐ בתוך כל פונקציה שצריכה אותם

ג ☐ חלק ב-`parser` וחלק ב-`transformer`

ד ☐ לא הגדרתי – כתבתי את המספרים ישירות בקוד

4. מה עושה פונקציית ה- `main()` / `process_pdf()` שלך?

א ☒ רק מחברת – קוראת לפונקציות אחרות, בלי לוגיקה עסקית בעצמה

ב ☐ עושה גם OCR, גם פרסור, גם שינוי הנתונים

יש לה +150 שורות כי "זה המקום הנוח" ג ☐

10 נקודות

+1 מבנה הקוד: מי יודע מה

21. איפה בקוד יש ידע על המבנה הספציפי של Type A לעומת Type B – כמה עמודות, מה הסדר שלהן, האם יש עמודת הפסקה?

א ☐ בכל מקום שצריך לטפל בנתונים – בלוגיקה, בפלט, בטסטים

ב ☒ רק בתוך הפרסרים – כל הקוד שאחריהם רואה רק AttendanceReport אחיד

ג ☐ בקובץ config משותף שכולם מייבאים

ד ☐ יש לי `if report_type == "TYPE_A"` בכמה מקומות בקוד

22. טקסט שמגיע מ-OCR מלוכלך: יש בו תווים שגויים, רווחים כפולים, תווים בלתי-נראים. איפה ומתי מנקים אותו?

א ☒ פעם אחת, מיד אחרי שמקבלים את הטקסט מ-OCR – לפני כל פרסור

ב ☐ כל פרסור מנקה בעצמו את הקלט שהוא מקבל

ג ☐ לא מנקים – מתמודדים עם הטקסט המלוכלך בכל פונקציה בנפרד

ד ☐ מנקים רק לפני כתיבת הפלט

15 נקודות

Domain Models – מבנה הנתונים 2

5. האם מחלקת השורה שלך (שמייצגת יום עבודה אחד) לא ניתנת לשינוי אחרי יצירה?

Immutability – אובייקט שלא ניתן לשנות

☒ א כן – משתמשת ב- `@dataclass(frozen=True)` או `NamedTuple`

☐ ב לא – `@dataclass` רגיל, אפשר לשנות שדות

☐ ג משתמשת ב-dictionary במקום מחלקה

6. איך את שומרת שעת כניסה (למשל 08:30) במחלקת השורה?

☒ א כ- `datetime.time` – מחרוזות מפוענחות רק פעם אחת ב-`parser`

☐ ב כמחרוזת "08:30" לאורך כל התוכנית

☐ ג כמספר שלם שמייצג דקות מחצות הלילה

7. מה קורה אם מנסים ליצור שורה שבה שעת יציאה קודמת לשעת כניסה?

☐ א לא בדקתי – יכול לקרות בשקט

☒ ב `__post_init__` זורק `ValueError` עם הסבר ברור

☐ ג בודקת בפונקציה נפרדת לפני שיוצרים את האובייקט

8. Type A ו-Type B צריכים לוגיקת טרנספורמציה שונה. איך טיפלת בזה?

א ☐ בתוך פונקציה אחת
if report_type == 'TYPE_A': ... else:
... ארוכה

ב ☒ Strategy Pattern: מחלקת בסיס אחת ושתי מחלקות ספציפיות
שמממשות אותה

ג ☐ שתי פונקציות נפרדות בלי יחסי ירשה/interface

9. (Factory Pattern) מי אחראי להחליט איזה פרסר להריץ לאחר שסיווגנו את הדוח?

א ☐ main.py בודקת את סוג הדוח
ויוצרת ישירות
TypeBParser() או TypeAParser()

ב ☒ פונקציית
get_parser(report_type)
שמחזירה את ה-
instance הנכון
כמו Factory

ג ☐ הפרסר בודק בעצמו מה סוג הדוח ומחליט מה לעשות

10. יש לך TypeAParser ו-TypeBParser שחולקים הרבה לוגיקה (ניקוי OCR, דילוג על שורות כותרת, בניית AttendanceReport). איך ביטלת את הכפילות?

א ☐ העתקתי את הקוד המשותף לשני הקבצים (copy-paste)

ב ☒ Template
BaseParser עם parse()
קבוע ו-abstract methods
שכל סוג מממש
:Method

פונקציות עזר גלובליות שכל הפרסרים קוראים להן א ג ☐

11. איזה סוגי שגיאות מוגדרים בפרויקט?

Custom
TransformationError , ClassificationError , ParseError exceptions א ☒
ייעודיים:

רק Exception / ValueError סטנדרטיים ב ☐

print("שגיאה") במקום raise ג ☐

20 נקודות

4 Dependency Injection ויכולת בדיקה

12. (Dependency Injection) איך TransformationService מקבלת את ה-strategy שלה?

יוצרת בעצמה TypeAStrategy() בתוך __init__ א ☐

מקבלת כפרמטר לconstructor – ברירת מחדל היא ה-registry האמיתי ב ☒

עושה import ישיר של הקבצים הספציפיים ג ☐

13. הטסטים שלך צריכים שיהיה מותקן Tesseract

(Tesseract = תוכנה שמחלצת טקסט מתמונות OCR – PDF)?

לא – מזריקים fake extractor שמחזיר טקסט קבוע, ללא OCR אמיתי א ☒

ב כן – הטסטים קוראים ל-OCR האמיתי ☐

ג אין לי טסטים ☐

14. אם תריצי את הפרויקט פעמיים על אותו PDF – תקבלי פלט זהה?

הנושא: Determinism – אותו קלט = אותו פלט

א לא – `random.random()` ישירות, בלי seed, אז התוצאה השתמשת ב- ☐

ב כן – כל שורה מקבלת מחולל אקראי `random.Random(20241007)` נפרד שמאותחל מהתאריך שלה: ☒

ג כן – כי קראתי `random.seed(42)` פעם אחת בהתחלה של התוכנית ☐

15. אם הטרנספורמציה של שורה אחת נכשלת – מה קורה?

א התוכנית קורסת עם traceback ☐

ב מוגדר log, השורה המקורית נשמרת כ-fallback, שאר השורות ממשיכות ☒

ג השורה הבעייתית מושמטת מהפלט בשקט ☐

15 נקודות

Testing 5

16. כמה unit tests יש בפרויקט שלך?

א ☐ אין טסטים

ב ☐ 4-1 טסטים

ג ☒ 5 טסטים ומעלה, בקבצים נפרדים לפי נושא

17. מה הטסטים שלך בודקים? (אפשר לסמן את התשובה הנכונה ביותר)

א ☒ Invariants: exit > entry, סכום שעות נוספות, determinism, שגיאה על קלט ריק

ב ☐ רק Happy Path – שהפונקציה מחזירה ערך כלשהו

ג ☐ בדקתי ידנית עם `print` ולא כתבתי טסטים אוטומטיים

18. יש לך integration test שבדק את כל הצינור מקצה לקצה?

א ☒ כן – עם `fake text_extractor`, בודק שנוצר קובץ פלט תקין

ב ☐ כן – אבל משתמש ב-PDF ו-OCR אמיתיים

ג ☐ לא, רק unit tests

10 נקודות

6 איכות קוד כללית

19. האם הפרויקט שלך ניתן להתקנה כ-package?

☒ א כן – יש pyproject.toml עם [project.scripts]

☐ ב יש רק requirements.txt

☐ ג אין קבצי הגדרה

20. איך את מגדירה את נתיב Tesseract בקוד?

Tesseract = התוכנה שמחלצת טקסט מהתמונה, צריך לדעת איפה היא מותקנת

`r'C:\Program Files\Tesseract-OCR\tesseract.exe'`

☐ א נתיב קשיח ישירות בקוד:

`os.environ.get("TESSERACT_CMD")`

☒ ב קוראת מ-environment variable

☐ ג לא הגדרתי – מניחה ש-Tesseract זמינה אוטומטית

עד 10 נקודות

בנוסף ★

סמני כל פריט שמימשת. כל פריט שווה 2 נקודות, עד 10 סה"כ.

✓ **CLI עם argparse** – הוספתי ממשק שורת פקודה שמאפשר להריץ את הכלי כך:

`python main.py report.pdf --html-only -o output/` במקום לערוך את הקוד בכל פעם. `argparse` = ספריית פייתון מובנית לפרסור פרמטרי שורת פקודה

✓ **Dockerfile** – יצרתי קובץ Dockerfile שמגדיר סביבה מלאה: Python + Tesseract

+ חבילת עברית, כך שכל אחד יכול להריץ את הפרויקט עם `docker run ...` ללא צורך להתקין שום דבר ידנית.

Docker = טכנולוגיה שאורזת תוכנה יחד עם כל התלויות שלה ל"קופסה" ניידת

Type Hints מלאים ✓

מקבלות ומחזירות, למשל: `def parse(text: str) -> AttendanceReport:` זה לא משנה את ריצת הקוד, אבל עוזר לקורא להבין מה הפונקציה עושה ומאפשר לכלים כמו mypy לגלות באגים לפני ריצה.

הוספתי סוג דוח חדש (Type C) בלי לשנות שום קוד קיים ☐

משתמשת ב- `import logging` (לא `print`) בכל השכבות ✓

ענית על כל השאלות? לחצי לקבלת הציון וההסברים

63

מתוך 110

סביר – יש מקום לשיפור

הבנת את הרעיון הכללי, אבל חלק מהעקרונות החשובים עוד לא יושמו. קראי את ההסברים.

פירוט לפי נושא:

20 / 20

מבנה הפרויקט וחלוקה לשכבות

10 / 0

מבנה הקוד: מי יודע מה

15 / 5

Domain Models

20 / 10

Design Patterns

20 / 15

Testability-I Dependency Injection

15 / 0

Testing

10 / 5

איכות קוד כללית

8+

★ בונוס

הסברים:

✓ שאלה 1: איך חילקת את הקוד שלך לקבצים? בחרי את התיאור שהכי מתאים לפרויקט שלך

נכון! 4 שכבות נפרדות = כל אחת אחראית על דבר אחד בלבד. שינוי בשכבה אחת לא שובר את השאר.

✓ שאלה 2: מי מייבא את מי בפרויקט?

מעולה! תזרים חד-כיווני הוא אחד הדברים החשובים ביותר בארכיטקטורה נקייה.

✓ שאלה 3: איפה מוגדרים המספרים של חוקי הטרנספורמציה (למשל: שעות עבודה מותרות, כמ

נכון! Single Source of Truth מונע באגים שנוצרים כשמשנים את הערך באחד מהמקומות ושוכחים את השאר.

✓ שאלה 4: מה עושה פונקציית ה-process_pdf() / main() שלך?

פרפקט! Orchestrator דק הוא סימן לארכיטקטורה בריאה.

✗ שאלה 21: האם הפרויקט שלך ניתן להתקנה כ-package?

pyproject.toml (או setup.py) הופכים את הפרויקט לחבילה שאפשר להתקין עם pip install -e. זה גם מאפשר להגדיר console_scripts כך שהפקודה 'attendance-report' עובדת מכל מקום. זה מה שמבדיל 'סקריפט' מ'package' אמיתי.

התשובה הנכונה: א

מובנות כמו חיסור. רק כשכותבים את קובץ הפלט (ה-renderer/generator), הופכים חזרה למחרוזת להצגה.
התשובה הנכונה: א

✓ שאלה 9: מה קורה אם מנסים ליצור שורה שבה שעת יציאה קודמת לשעת כניסה?

נכון! __post_init__ עם raise ValueError הוא המקום הנכון לאמת נתוני דומיין.

✓ שאלה 10: Type A ו-Type B צריכים לוגיקת טרנספורמציה שונה. איך טיפלת בזה?

Strategy Pattern – בדיוק! מאפשר החלפת אלגוריתם בזמן ריצה ללא if/else.

✗ שאלה 11: (Factory Pattern) מי אחראי להחליט איזה פרסר להריץ לאחר שסיווגנו את הדו

Factory Method מתאים כאן: פונקציה כמו get_parser(report_type) מחזירה instance של הפרסר הנכון. הקוד שקורא לה לא צריך לדעת שקיימים TypeAParser ו-TypeBParser. להוסיף Type C = שורה אחת ב-registry, שום דבר אחר לא משתנה.
התשובה הנכונה: ב

✓ שאלה 12: יש לך TypeAParser ו-TypeBParser שחולקים הרבה לוגיקה (ניקוי OCR, דילוג

Template Method – נכון! הבסיס מגדיר מתי, הירש מגדיר איך.

✓ שאלה 13: איזה סוגי שגיאות מוגדרים בפרויקט?

מעולה! Custom exceptions הן חלק מה-API הציבורי של המודול.

✓ שאלה 14: (Dependency Injection) איך TransformationService מקבלת את ה-strategy ש

Dependency Injection דרך constructor – הכי נכון! מאפשר הזרקת stubs בטסטים.

✗ שאלה 15: הטסטים שלך צריכים שיהיה מותקן Tesseract = תוכנה שמחלצת טקסט

OCR (Optical Character Recognition) הוא התהליך שבו הספרייה Tesseract סורקת תמונה של PDF ומחלצת ממנה טקסט. הבעיה: Tesseract הוא תוכנה חיצונית שצריך להתקין על המחשב, הריצה שלה אורכת כמה שניות, ובסביבת CI (שרת אוטומטי שמריץ טסטים) היא לפעמים לא מותקנת. לכן הטסטים לא צריכים להיות תלויים בה. הפתרון: הפונקציה process_pdf() מקבלת פרמטר text_extractor שאפשר להחליף. בטסט, במקום OCR אמיתי, מעבירים פונקציה פשוטה שמחזירה טקסט מוכן.
התשובה הנכונה: א

X שאלה 16: אם תריצי את הפרויקט פעמיים על אותו PDF – תקבלי פלט זהה? הנושא:**Determi**

הנושא נקרא Determinism – אותו קלט תמיד מייצר אותו פלט. בפייתון, `random.random()` מייצר מספר אקראי שונה בכל פעם – זה לא `random.seed(42)` deterministic. בתחילת התוכנית קובע נקודת התחלה אחת לכל הרצה, אבל הבעיה שכל שינוי קטן בקוד (הוספת עוד קריאה ל-`random` במקום אחר) ישנה את כל התוצאות. הפתרון הנכון: לכל שורה יוצרים מחולל אקראי נפרד שמאותחל מהתאריך שלה: `rng = random.Random(20241007)` עבור שורה מ-07/10/2024. כך כל שורה תמיד מקבלת אותה "הטיה" ללא תלות בסדר או בכמות הקריאות במקומות אחרים.

התשובה הנכונה: ב

X שאלה 17: אם הטרנספורמציה של שורה אחת נכשלת – מה קורה?

כשה-`transformer` נכשל בשורה אחת (`TransformationError`), ה-`service` לא צריך לקרוס לגמרי. הפתרון: `try/except` בלולאה, שמור את השורה המקורית כ-`fallback`, המשיך לשאר השורות, ולסיום החזר `ProcessingResult` שמכיל גם פלט גם רשימת שגיאות. המשתמש רואה מה נכשל בלי שהתוכנית קרסה.

התשובה הנכונה: ב

X שאלה 18: כמה unit tests יש בפרויקט שלך?

המינימום הסביר לפרויקט כזה הוא +5 טסטים: לפחות מודלים (ולידציה), `classifier`, `parser`, `strategy (determinism + overtime)` ו-`integration`. פחות מזה = אין ביטחון שהדבר עובד.

התשובה הנכונה: ג

✓ שאלה 19: מה הטסטים שלך בודקים? (אפשר לסמן את התשובה הנכונה ביותר)

נכון! `invariants` הם הדבר הכי חשוב לבדוק – הם מה שמבדיל בין 'עובד במקרה' לבין 'עובד תמיד'.

X שאלה 20: יש לך integration test שבדוק את כל הצינור מקצה לקצה?

`Integration test` צריך לבדוק את כל הצינור ביחד – אבל בלי OCR אמיתי. מזריקים `text_extractor` פיקטיבי שמחזיר טקסט קבוע, ובודקים: הדוח סווג נכון, נוצר קובץ HTML, הקובץ מכיל תאריכים. זה בודק ש-'החיבורים' עובדים בלי לצאת למערכת קבצים אמיתית.

התשובה הנכונה: א

שלחי את התוצאות למרצה:

=== PDF Attendance Report – Self Assessment ===

Student: אילה הרדי

GitHub: <https://github.com/AyalaOnGit/Attendance-Report-Gen/tree/upgrade>

Date: 2.5.2026, 23:01:49

Score: 63/100 (+8 bonus)

--- Section Scores ---

מבנה הפרויקט וחלוקה לשכבות: 20/20

מבנה הקוד: מי יודע מה: 0/10